

Engine Architecture

Engine Architecture I

The engine is nine small modules, each with one responsibility.

`geometry` is pure arithmetic over points: the page trim, its margins, and a `ColumnGrid` that partitions a region into equal columns with gutters and reports the centre of each gutter (where a hairline rule is drawn). It imports no rendering library, which makes the column mathematics trivially testable.

`content` defines the typed object graph — `Edition`, `Page`, `Story`, `Box`, `Figure`, `Block` — and the strict YAML loaders that build it. The loaders are forgiving about *shape* (a body element may be a bare string or a tagged mapping) but strict about *structure*: an unknown page template or box kind raises an error naming the offending value and the allowed set.

`config` is the strict render configuration (trim size, default column count, rail width, gutter), validated the same way.

Engine Architecture II

typography owns every font name: it registers the Didot display, Georgia text and Helvetica label faces (with a base-14 fall-back when the system lacks them) and builds the paragraph stylesheet that the rest of the engine draws from through named `Fonts` roles.

figures is the only module that *generates* imagery: the halftone harbour, lighthouse and redwood engravings (`Pillow`), the weather, tide and crab-landings charts (`Matplotlib`), and the colour display-ad graphics. It writes deterministic PNGs that `components` later places.

`components` builds the flowables that live *inside* a column: a story's headline, byline and body; a raised drop cap; a pull quote; a ruled box; a data table; a figure with its cutline. `furniture` draws the elements that are *placed* rather than flowed: the nameplate and its ears, the section banner, the folio, the hairline column rules, and the spanning lead headline.

Engine Architecture III

layout is the bridge. For each page it draws the furniture (which returns the y-coordinate where the column grid begins), constructs the column frames — including an optional narrow rail and the room reserved for a spanning headline — and pours the flowables into the frames with ReportLab's `Frame.addFromList`, which splits paragraphs across columns automatically. Content that does not fit is returned and reported as an *over-set* page rather than silently dropped.

engine ties it together: register fonts, build the stylesheet, open a canvas at the configured trim, render each page, and write the PDF plus a machine-readable render report.

Engine Architecture IV

This **drawn-furniture / flowed-body** split is the key architectural choice. A pure document-template approach cannot place a headline that spans several columns above body text that flows beneath it; drawing the furniture first, and beginning the column frames below it, makes spanning headlines, standing boxes and automatic text flow coexist on the same page.

Methods I

Producing an edition is a deterministic, four-step method, run by the pipeline and covered end-to-end by the test suite:

1. **Load** — content parses `edition.yaml` and every `content/pages/*.yaml` into the typed Edition graph, rejecting unknown page templates or box kinds by naming the offending value and the allowed set.
2. **Measure** — geometry derives the trim, margins and `ColumnGrid` for each page from the strict config, independently of any rendering library.
3. **Compose** — components and furniture build the flowed and drawn elements; layout pours them into the column frames and reports any content that does not fit as an over-set page rather than dropping it silently.
4. **Emit** — engine registers fonts, renders each page to the canvas, and writes the PDF alongside a machine-readable render report.

Methods II

Every step is pure-data-in, artifact-out and seeded deterministically, so the same edition manifest always yields a byte-identical paper — the reproducibility contract verified in `tests/` and described in `04_reproducibility.md`.